First steps with Logic Simulation

For a visual demonstration of the material in this document, please see Tcl tutorial at:
https://youtu.be/r6p5mCPh4yc

1 Introduction

In the lecture, we overviewed the Verilog Hardware Description Language (HDL), saw the basic syntax, constructs, and recommended coding style. In fact, you probably realized that it's not that hard to write Verilog, since there are very few commands and reserved words that you are actually allowed to use. On the other hand, it's not only much harder, but also critical, to ensure that what you coded actually works.

In this document, we will learn how to simulate a Verilog Testbench, implementing a directed test that was written in order to verify the correct functionality of a digital module. This will be done with Cadence's Incisive environment.

1.1 Cadence Xcelium (formerly Incisive)

As with all Electronic Design Automation (EDA) tools, logic simulation can be achieved with products by various vendors, most often including "the big three" (Cadence, Synopsys and Mentor Graphics). In this course, we will be using Cadence's simulation environment, which is distributed as part of the Incisive package of verification tools. The EDA vendors tend to confuse us with their tool names, which makes it extremely difficult to find solutions for problems or understand how to actually do what we want to do, so I will give you a few starting points.

Logic simulation requires various steps, such as parsing the syntax of the Register-Transfer Level (RTL) code and testbenches, elaboration and binding of the modules, running the simulation, and displaying graphical waveforms. The Incisive family of tools combines all of these under one hood (this may seem trivial, but it's a rather new concept). In fact, the majority of the tools are now initiated with a single executable called `xrun` (formerly `irun`) which further simplifies or complicates things, depending on who you ask. Why does it complicate things? Because actually, the original tools (such as `ncvlog` and `simvision`) still exist and still have their own set of options, but you can apply these through the `xrun` command and shell. Easy or completely crazy? You decide.

1.2 The xrun executable

The `xrun` command does the following:

1. **Compiles** the code from languages, such as verilog, VHDL, Specman e, C++, etc.
   For example, if you passed a verilog file, the `ncvlog` tool will be executed for compilation.
2. **Elaborates** the design with the `ncelab` tool.
3. **Simulates** the testbench with the `ncsim` tool.

This is instead of running each tool separately. All command line options will be passed to the appropriate tool, so you can do whatever you had originally done, if you previously used each tool by itself.

Just to understand a bit of what's going on, when you run `xrun`, a lot of scratch directories for all the simulation files are created in your workspace with symbolic links to the current scratch directories. In addition, three files are created in the run directory:

- `xrun.log`: The logfile with all errors and warnings. Use this to debug your run (sorry that it looks like it's in some gibberish language… You'll get used to it…)
- `xrun.history`: The last command line used to invoke Xcelium

```
%> cat xrun.history
xrun -f ../scripts/xrun_options.rtl
```

- `xrun.key`: The last commands run inside the Xcelium environment after invocation

```
%> cat xrun.key
database -open waves -into waves.shm -default
probe -create -shm sm_tb.clk sm_tb.rst_n sm_tb.up_dwn_n sm_tb.act
                              sm_tb.count sm_tb.ovflw
run 5000
exit
```

Note that if `xrun` is *reinvoked*, it will check if the source files have been modified and will only recompile the ones that have changed. If nothing has changed, `xrun` will go directly to simulation.

1.3 Running Xcelium

As mentioned above, to run Xcelium, all you really need is the `xrun` command… and a few billion options. But in general, if you want to run a simulation on a Testbench module named `my_testbench` with a `.v` file of the same name, just try:

```
%> xrun my_testbench.v
```

But actually, you will need many more options. Here are some of the really necessary ones:

- `<filename>.v` : This is a verilog file or list of verilog files to compile. They will be stored under a scratch directory called `worklib`.
- `-access` : provide `ncelab` with read access to simulation objects.
- `+rw` : provide read/write access to simulation objects
- `-debug` : This is the same as using the `-access +rw` options.
- `-gui` : opens the graphical interfaces of `SimVision` to see waveforms and debug.
- `-y <directory>` : compiles verilog files in the directory you put here. These files will be saved under a scratch directory of the same name.
- `-v <file.v>` : Specify the name of the library file to use.
- `-clean` : Delete the INCA_libs directory before executing.
- `-compile` : Parse/compile the source files, but do not elaborate.

- `-elaborate` : Parse/compile the source files, elaborate the design, and generate a simulation snapshot, but do not simulate.
- `-f <file.args>` : pass a file of arguments to `xrun`. This file can contain, for example, a list of verilog files to compile.
- `-input <file.tcl>` : pass a TCL file to run after elaboration.

Commonly used flags are:

- `-sv`: Accept **System Verilog** code (belongs to the **xmvlog** tool)
- `-v93`: Accept VHDL 93 updates (belongs to the **xmvhdl** tool)
- `-debug`: Saves data base info for displaying waveforms and debugging.
- `-gui`: Opens up the **Simvision** gui.
- `-timescale`: Adds a default `timescale directive for Verilog modules that are missing them.
- `-define`: Define a value to send to the Verilog code (belongs to the **xmvlog** tool)
- `-nospecify`: Disregards any annotated delays (do not use this option with SDF backannotation!)

1.4 Passing a list of files (and command) to Xcelium

Most designs have a number (maybe, a large number) of files that are needed for design elaboration. At the very least, you almost certainly need to read in a testbench module that instantiates an RTL module, and these are in separate files (if you go by the design style guidelines in Lecture 2 !!!). You can `` `include `` the files in the testbench or you can list all the files after the xrun command, but this quickly becomes very long.

A better way is to create a list of files for simulation and using the `-f` argument:

```
~> xrun -f ../sourcecode/rtl_files.list
```

Inside the example file `rtl_files.list`, you would have a list of files to read, relative to your run directory:

```
# rtl_files.list
../sourcecode/tb/mydesign_tb.v
../sourcecode/rtl/mydesign.v
```

Actually, you can put a long list of options inside a `-f` file. What Xcelium actually does is it copies the content of the `-f` file onto the run command (you will see this inside the `xrun.log` file), so you can add all of your flags and such into one file and just run the `xrun` command with a single `-f` script.

For your convenience, I have created a file under the `scripts` directory called `xrun_options.rtl`. This file has typical flags, so you can edit it and just run

```
xrun -f ../scripts/xrun_options.rtl
```

and you're all set.

2 Getting help

As with most tools, just use the `-h` option to get the main options:

```
%> xrun -h
```

Since `xrun` combines many tools, you have to ask for particular help for the option of a certain tool (yes, I know this is complicated, but bear with me). So to find out what the tools that you can get help for are use the `-helpshowsubject` option:

```
%> xrun -helpshowsubject
```

Now, you can ask for help about one of these tools (or subjects) using the `-helpsubject` option. For example, to get help with the verilog compiler (`ncvlog`):

```
%> xrun -helpsubject ncvlog
```

You can also use the `nchelp` utility to get help on tool messages.

```
%> nchelp xrun BDOPT
%> nchelp ncvlog NOBIND
%> nchelp ncsim NOSNAP
```

To get a general list of all the help options, use the `-helphelp` option:

```
%> xrun -helphelp
```

3 Preparing your code for simulation

In this section, we will prepare our code for simulation. You are free to use whatever text editor you would like for editing your RTL; however, be warned! Real engineers use either `VI` or `eMacs`. If you go into an honorable workplace and open the dreaded `gedit`, you will not be taken seriously! And there are good reasons for this. I highly recommend getting used to `VI` or `eMacs` at this early stage in your VLSI career. It may be a bit ugly at first, but believe me – you won't regret it!

Note that another option is to use VSCode, which is a recent IDE for editing code provided as open source by Microsoft. To run VSCode, just run `code` in a terminal.

3.1 RTL Preparation

In this exercise, we will take the FSM from the lecture and use it as our toy example. First, open a new file in the graphical-visual `VI` editor (`gvim`):

```
%> gvim sm.v
```

Now write your module in the file and save it.

3.2 Customizing VI

I know, I know. You can't get used to VI, since it doesn't accept your usual Windows shortcuts, like `CTRL-C`, `CTRL-V`.

Alas! **VI** is not as dumb as you think. EVERYTHING is customizable. And luckily, some other Windows user wrote a configuration file to fix our biggest worry. To add some configuration to **GVIM**, edit (or create) a file called `.gvimrc` in your home directory:

```
%> gvim ~/.gvimrc
```

Now, anything you find on Google for customizing **GVIM** can go straight into this file. But specifically for Windows shortcuts, just add the following line:

```
source $VIMRUNTIME/mswin.vim
```

Now restart **GVIM** and try to copy, paste, save, undo. COOL!

Note that if a veteran VI user is trying to help you and is getting frustrated because "nothing works!", it's probably due to what we just did. To really become a VI expert, learn the classic keyboard shortcuts.

4 Running a Simulation

Ready to run your simulation?

```
%> xrun -debug -gui sm_tb.v sm.v
```

And then after compilation and elaboration are successful, add your signals to the waveform window and type:

```
%> run 1000ns
```

And start playing around from here.

Remember what I wrote above – you can put all the command line option inside a `-f` file (or several `-f` files), which makes running the tool a bit more convenient.

5 Automating your Incisive Run

In general, Incisive has a full **TCL** shell with commands that can automate everything. Since we like to debug using the waveform windows, we often find ourselves using the GUI and clicking all kinds of buttons that "magically" do things. As you will see, with other tools, you usually just source **TCL** scripts, rather than using GUI controls. At least for setting up your run, it is very useful to do this with **Xcelium**, as well.

5.1 Saving your Waveform Window Setup

One of the recurrent operations you do when running directed tests is to open the waveform window, add signals, change the Radix of the signals, etc. Well, guess what. You actually only have to do this once and you can have it occur automatically from then on!

Assuming you have set up your **Simvision** waveform window, by clicking on `Add Signals to Waveform`, you now have a window that looks something like this:

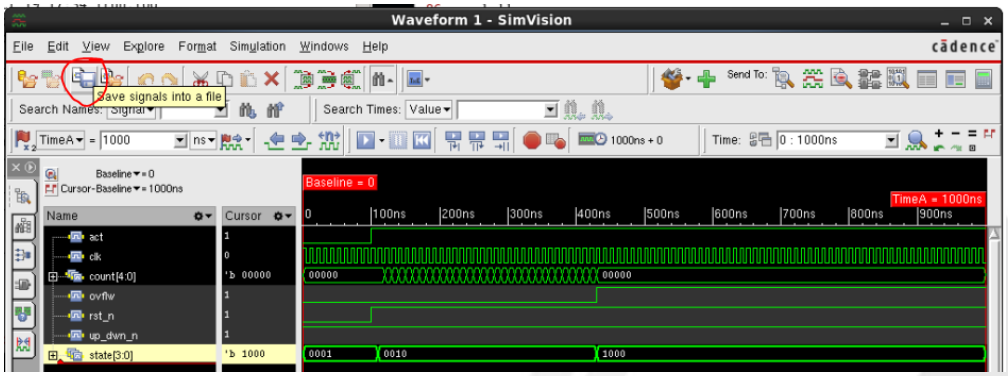

Figure 1: Waveform Window with Selected Signals

As you can see, the circled icon is for `save signals into a file`. Well, go ahead and save them!

Now, next time you load Incisive, you can just select the icon right next to it (`load signals from a file`) and *presto*, your signals will appear as you had set them up last time.

> Note that the saved signals definition file is just a TCL script, so you can open it and edit it to change around the signal order, properties, etc. Find out what all the commands do using the Incisive Command Reference or other Cadence Help methods.

5.2 Creating a run script

In the previous subsection, we created a waveform description command file and loaded it into **Incisive** on our next run. But this is just a particular use case of generally running a script with predefined commands in **Incisive**.

You can run a script file in two primary ways:

1. Source the script file from the **ncsim** or **SimVision** command line:

```
ncsim> source ../scripts/simulation.tcl
```

2. Automatically run your command file when **Xcelium** starts:

```
~> xrun my_tb.v -f rtl_files.list -debug -gui \
        -input ../scripts/simulation.tcl
```

Probably the easiest way to figure out what to put in your **TCL** script is to see what you did manually, by saving your current session as a script. This is done by choosing `File → Save Command Script` from the **SimVision Design Browser** menu:

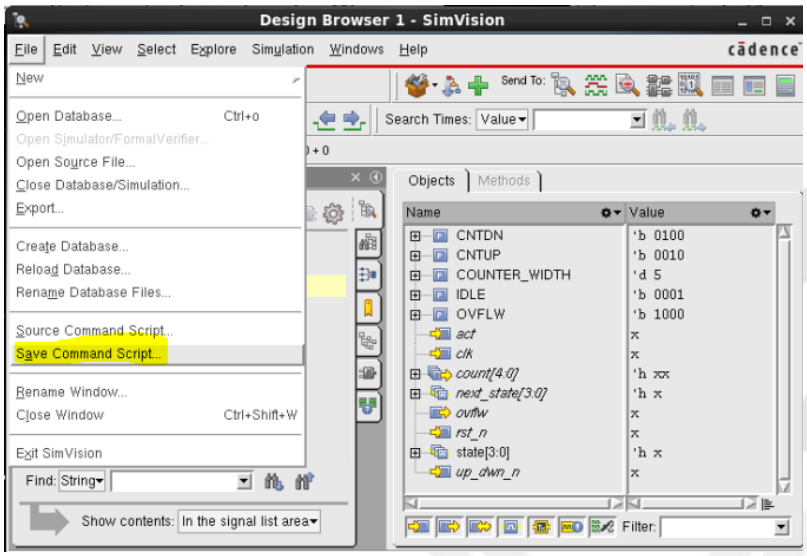

Figure 2: Save Command Script

Now you can edit the saved file to automate your simulation run.

6 Gate-level Simulation

What we have run so far is logic simulation on RTL. This is based on behavioral Verilog (or other HDL) code with zero delay annotation. However, after you have run through synthesis, you no longer have behavioral RTL code, but rather you are left with a gate-level netlist (GTL or GLV) that is made up of library components (standard cells and other IOs). Following synthesis (and later in the flow), you will want to verify that the functional tests still work, and therefore, you will want to run your test bench on the netlist.

6.1 What are the main differences between RTL and GLV simulation?

- **No parameters**: During synthesis, all parameters have been evaluated and turned into actual nets and gates.
- **Standard cell instantiation**: During synthesis, behavioral code is mapped onto standard cells, so your netlist is built up of instantiations of actual library gates rather than generic HDL code.
- **Delay Annotation**: While everything in RTL is ideal (=zero delay), some standard cells may have some default delay in their behavioral models and you can back-annotate timing arcs onto the cells, so you may have to deal with timing.
- **IO Delays**: Something that can happen during RTL simulation, as well, but tends to show up at gate level – and especially with annotated timing – is problems with interface delays. If you toggle your inputs or sample your outputs at the same time as a (rising) clock edge, all hell can break loose.
- **Complexity**: While you can simulate things like arithmetic and wide busses very easily in RTL, once you go to gate level, you have actual nets and gates. This makes the simulation much more complex and can lead to extremely long runtimes. Therefore, GLV simulation is used sparingly and not run thoroughly to achieve high coverage.

6.2 Setting up GLV simulation

Based on the differences from RTL simulation, described above, for GLV simulation, you generally need to:

- **Clean out parameters**: Your testbench should no longer use parameters to instantiate or communicate with your synthesized netlist. You can do this with a simple define:

```
`ifdef RTL
    dut #(WIDTH) DUT (.clk(clk),.rst_n(rst_n),.in(in),.out(out));
`endif
`ifdef GATE_LEVEL
    dut DUT (.clk(clk),.rst_n(rst_n),.in(in),.out(out));
`endif
```

  Don't forget to add `-define GATE_LEVEL` to your `-f` file!

- **Standard Cells**: Add behavioral model of Standard Cell (and other libraries) to your `-f` file:

```
-f ../libraries/librarie.my-standard-cells.f
```

- **Delay Annotation**: To make sure the behavioral models aren't messing around, add the `-nospecify` flag to the command line:

```
xrun -nospecify
```

- **IO Delays**: Make sure you change your inputs at a phase shift from your rising clock edge. To make this more robust, toggle your inputs at the falling edge of the clock. The same is for a checker sampling outputs – make sure they don't check the output at the same moment they may have changed or not with zero delay. However, note that for both input and output delays, after applying timing backannotation, they need to meet the STA constraints of your SDC!

6.3 Timing backannotation

Now that you have a running zero-delay gate level simulation, why not check that it works with real timing arcs coming from the synthesis or place and route run?

6.3.1 SDF File

An SDF file contains the timing information. You can export it from your synthesizer or place and route tool.

```
write_sdf <design_name>.sdf
```

6.3.2 Testbench Annotation

We have to now add SDF information to the testbench (Verilog file):

```
initial
   begin
     $sdf_annotate(<design_name>.sdf,<DUT>,,"sdf.log" ,"MAXIMUM");
   end
```

The parameters given to the `$sdf_annotate` command are important:

1. **Name** of the `.sdf` file
2. The "**scope**" of the `.sdf` file. This is the name of the instance inside the testbench that the `.sdf` file relates to. In most cases this will be the instantiation name of the `Verilog` netlist inside the testbench, but you can actually take several `.sdf` files and map them to several instantiations inside the testbench.

Note that the scope is the first place to check if your annotation is not working!
Remember that I told you so…

3. Leave it empty – there are two *commas* there on purpose!
4. Name of the **logfile**.
5. The **corner** to take from the `.sdf` file (remember, there is a tuple of `(worst:typical:best)`)

6.3.3 Run Xcelium with backannotation

To get the simulator to report the SDF backannotation and help debug:

```
xrun +sdf_verbose -sdfstats sdf_stats.txt
```

- `+sdf_verbose` : writes out the sdf logfile .
- `-sdfstats` : writes out the sdf statistics file.

6.3.4 Checking that the SDF Backannotation worked

Now open the log files that the SDF annotation outputs and see that everything looks okay:

- `sdf.log` : It should show numbers annotated on every instance of the design. If everything is 0.000 then you should be worried…
- `sdf_stats.txt` : This file shows a list of unannotated paths. It shouldn't be too long. Actually, I'm not entirely sure why it has any entries at all, but certain things do not get annotated by **Innovus**, it seems.

A very important thing is to check delays in the waveforms. If zero delays are applied, like in RTL simulation, then something is wrong. If you have a delay through each logic gate, that is a good sanity check to assume that everything is okay.

Figure 3: SimVision waveform with delays due to SDF back-annotation

This document was provided to you courtesy of Prof. Adam Teman of the EnICS Labs Institute in the Alexander Kofkin Faculty of Engineering at Bar-Ilan University. The content was entirely written and compiled by Prof. Teman (without the help of AI tools), based on knowledge and non-proprietary data. You can find all of Prof. Teman's lectures and materials at the EnICS Labs website: https://enicslabs.com/education/

For any comments or enquiries, Prof. Teman can be reached at adam.teman@biu.ac.il